

CrowdCall Flyer Creator

Comprehensive Improvement Suggestions

This document contains detailed improvement suggestions for the CrowdCall Flyer Creator React + TypeScript + Vite application. Recommendations are organized by category and prioritized based on impact and effort.

■ Type Safety & ESLint Configuration

1. Enable Type-Aware ESLint Rules

Your current ESLint config uses basic rules. Upgrade to type-aware linting by modifying `eslint.config.js` to use `tseslint.configs.strictTypeChecked` or `recommendedTypeChecked`. This will catch more type-related bugs during development.

2. Fix Missing Dependencies in `useEffect/useCallback`

Several hooks have missing dependencies that could cause stale closures. Specifically: `flyerStore.tsx` (undo/redo callbacks depend on `state.content`), `App.tsx` (media query listener). Add all required dependencies to avoid subtle bugs.

■■ Architecture & State Management

3. Consider Zustand or Redux Toolkit

The custom Context + `useReducer` pattern has limitations: no middleware support, manual history management, and difficulty testing. Migrate to Zustand for simpler state management with built-in devtools and persistence middleware.

4. Improve History Management

Current issues: `structuredClone` may fail with complex objects, no debouncing on history pushes, undo/redo doesn't work well with image uploads. Add debouncing (300ms) and use JSON serialization for safer state cloning.

5. Separate Concerns in Store

The store handles too much: state management, `localStorage` persistence, keyboard shortcuts, and history tracking. Extract keyboard shortcuts to `useKeyboardShortcuts.ts` and persistence to a separate utility module.

■ Component Improvements

6. ControlPanel - Reduce Duplication

AdvancedPanel duplicates ColorPicker and QR code logic from ContentTab/DesignTab. Create reusable sub-components: QRCodePanel.tsx, FooterToggle.tsx, ColorSection.tsx.

7. MobileControlSheet - Memory Leak Risk

Create a reusable useMediaQuery hook to handle media query listeners consistently across the application. This prevents memory leaks and reduces code duplication.

8. FlyerCanvas - Template Navigation

Memoize current template index calculation to prevent unnecessary re-renders when navigating between templates using prev/next buttons.

9. LayoutComponents.tsx - Extract Common Logic

This 36KB file contains 12 layout components sharing similar patterns. Create a base layout component or render helpers to reduce duplication and improve maintainability.

■ Performance Optimizations

10. Memoize Expensive Computations

Memoize template and dims calculations in FlyerCanvas.tsx. Consider React.memo() for layout components to prevent unnecessary re-renders.

11. Lazy Load Modals

Use React.lazy() for TemplateModal and DownloadModal to reduce initial bundle size. Modals are conditionally shown but always rendered currently.

12. Optimize Image Handling

Add image compression before storing in state, consider IndexedDB for large images instead of localStorage, and add loading states for image uploads.

■ Testing Gaps

13. Add Unit Tests

Set up Vitest + React Testing Library. Test critical paths: undo/redo, template application, color changes. Mock localStorage for store tests.

14. Add E2E Tests

Use Playwright or Cypress for critical user flows: creating flyers, applying templates, downloading/exporting, mobile responsive behavior.

■ Accessibility Improvements

15. ARIA Labels & Keyboard Navigation

Add focus trap in modals, escape key handling, skip links for keyboard users, and ensure all interactive elements have visible focus indicators.

16. Color Contrast

Some color combinations may fail WCAG AA standards. Use the `isLightColor` utility more consistently to ensure readable text.

■ Code Organization

17. Restructure Components Folder

Reorganize into: `ui/` (shadcn primitives), `panels/` (ControlPanel sections), `modals/`, `canvas/` (FlyerCanvas, layouts), `layouts/` (Flyer layout components).

18. Export Barrels

Add `index.ts` files for cleaner imports. Example: `src/components/ui/index.ts` exporting all UI components.

■ Build & Dev Experience

19. Remove Debug Plugin

Ensure `kimi-plugin-inspect-react` is dev-only by conditionally including it based on `NODE_ENV` environment variable.

20. Add Bundle Analysis

Install `rollup-plugin-visualizer` to identify bundle bloat and optimize dependencies.

21. Consider SWC Plugin

Switch to `@vitejs/plugin-react-swc` for faster builds and better performance.

■ Bug Fixes

22. localStorage Error Handling

Add versioning for stored data schema, migration logic for breaking changes, and user notification when save fails due to quota exceeded.

23. Template Application Race Conditions

When applying templates rapidly via prev/next buttons, state updates may conflict. Add cancellation or queuing mechanism.

24. Zoom Level Persistence

Zoom level resets on refresh. Consider persisting user preferences in localStorage.

■ Mobile UX Enhancements

25. Touch Gestures

Add pinch-to-zoom for canvas, swipe gestures for template navigation, and ensure minimum 44px touch targets.

26. Bottom Sheet Improvements

Add snap points for different panel heights, prevent scroll bleed-through when sheet is open, and add haptic feedback on tab switches.

■ Priority Recommendations Summary

Priority	Task	Effort	Impact
■ High	Enable type-aware ESLint	Low	High
■ High	Fix missing hook dependencies	Low	High
■ High	Add proper error boundaries	Medium	High
■ Medium	Extract reusable panel components	Medium	Medium
■ Medium	Add unit tests for store	Medium	High
■ Medium	Optimize image handling	Medium	Medium
■ Low	Lazy load modals	Low	Low
■ Low	Add bundle analyzer	Low	Medium

These recommendations are based on a thorough analysis of the codebase. Implementation should follow the priority order, starting with high-impact, low-effort items. Regular code reviews and testing will help maintain code quality as the application evolves.